

▼ Prática de Clusterização - Online Retail

Baseado no exemplo de Manish Kumar (<https://www.kaggle.com/hellbuoy/online-retail-k-means-hierarchical-clustering>)

O Online Retail (<https://www.kaggle.com/lakshmi25npathi/online-retail-dataset>) é um conjunto de dados transnacional que contém todas as transações ocorridas entre 01/12/2010 e 09/12/2011 para um varejo on-line registrado e não registrado no Reino Unido. A empresa vende principalmente presentes exclusivos para todas as ocasiões. Muitos clientes da empresa são atacadistas.

As etapas são amplamente divididas em:

- Etapa 1: Leitura e compreensão dos dados
- Etapa 2: limpeza de dados
- Etapa 3: preparação de dados
- Etapa 4: construção de modelo
- Etapa 5: Análise Final

Boa atividade!

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import datetime as dt

%matplotlib inline
```

▼ Etapa 1: Leitura e compreensão dos dados

```
retail = pd.read_csv('online-retail-dataset.csv', sep=";", encoding="ISO-8859-1", header=0)
retail.head()
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID
0	536365	85123A	WHITE HANGING HEART T- LIGHT HOLDER	6	01/12/2010 08:26	2,55	17850.0
1	536365	71053	WHITE METAL LANTERN	6	01/12/2010 08:26	3,39	17850.0
			CREAM CUPID		01/12/2010		

```
print("quantidade de registros: %d" % retail.shape[0])
```

```
quantidade de registros: 541909
```

```
retail.info()
```

```
retail.describe()
```

	Quantity	CustomerID
count	541909.000000	406829.000000
mean	9.552250	15287.690570
std	218.081158	1713.600303
min	-80995.000000	12346.000000
25%	1.000000	13953.000000
50%	3.000000	15152.000000
75%	10.000000	16791.000000
max	80995.000000	18287.000000

▼ Etapa 2: Data Cleaning

Nesta etapa identificaremos os registros nulos ou que possuem alguma inconsistência

▼ Identificando e excluindo os registros com atributos nulos

```
df_null = round(100*(retail.isnull().sum())/len(retail), 2)
df_null
```

```
InvoiceNo    0.00
StockCode    0.00
Description   0.27
Quantity     0.00
```

```
InvoiceDate    0.00
UnitPrice      0.00
CustomerID     24.93
Country        0.00
dtype: float64
```

```
retail = retail.dropna()
```

```
print("quantidade de registros: %d" % retail.shape[0])
```

```
quantidade de registros: 406829
```

▼ Etapa 3: Preparação dos dados

Nesta etapa realizaremos a conversão de tipos, construção dos atributos, exclusão de amostras expurias e transformação de escala dos dados

▼ Conversão do tipo de dados do atributo UnitPrice e do InvoiceDate

1. Conversão do tipo do atributo InvoiceDate para datetime
2. Conversão do tipo do atributo UnitPrice para ponto flutuante
3. Conversão do tipo do atributo CustomerID para texto

```
# convertendo o campo CustomerID para string
retail['CustomerID'] = retail['CustomerID'].astype(str)

#utilizamos uma função do pandas para realizar a conversão para o formato data
retail['InvoiceDate'] = pd.to_datetime(retail['InvoiceDate'],format='%d/%m/%Y %H:%M')

#alteramos o sinal de casas decimais de virgulta para ponto
retail['UnitPrice'] = retail['UnitPrice'].apply(lambda x: x.replace(',','.'))
#utilizamos uma função do pandas para realizar a conversão para ponto flutuante
retail['UnitPrice'] = pd.to_numeric(retail['UnitPrice'])

retail.info()
```

▼ Criando campos para o agrupamento (Feature Engeneering)

Para realizar o trabalho, nos ainda criaremos três campos. Esse processo é chamado feature engineering. Os campos que criaremos nos ajudarão a compreender o comportamento dos clientes. São eles:

1. R (Recencia): número de dias desde a última compra
2. F (Frequência): Número de compras
3. M (Faturamento): Faturamento total do cliente

▼ Criando o atributo Faturamento total do cliente

```
# criamos um novo campo com o total de cada compra
retail['Amount'] = retail['Quantity']*retail['UnitPrice']
# fazemos o agrupamento das compras por CustomerID
rfm_m = retail.groupby('CustomerID')['Amount'].sum()
rfm_m = rfm_m.reset_index()
rfm_m.head()
```

	CustomerID	Amount
0	12346.0	0.00
1	12347.0	4310.00
2	12348.0	1797.24
3	12349.0	1757.55
4	12350.0	334.40

▼ Criando o atributo frequencia de compras realizadas pelo cliente

```
rfm_f = retail.groupby('CustomerID')['InvoiceNo'].count()
rfm_f = rfm_f.reset_index()
rfm_f.columns = ['CustomerID', 'Frequency']
rfm_f.head()
```

	CustomerID	Frequency
0	12346.0	2
1	12347.0	182
2	12348.0	31
3	12349.0	73
4	12350.0	17

▼ Criando o atributo de Recência de Compras

```
# selecionamos a última compra de toda a base
max_date = max(retail['InvoiceDate'])
# Criamos um atributo que identifica o tempo em dias desde a compra mais recente da base
retail['Diff'] = max_date - retail['InvoiceDate']
retail.head()
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID
0	536365	85123A	WHITE HANGING HEART T- LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850.0
1	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850.0
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850.0

```
# pegamos a compra mais recente de cada cliente
rfm_p = retail.groupby('CustomerID')['Diff'].min()
rfm_p = rfm_p.reset_index()
rfm_p.head()
```

	CustomerID	Diff
0	12346.0	325 days 02:33:00
1	12347.0	1 days 20:58:00
2	12348.0	74 days 23:37:00
3	12349.0	18 days 02:59:00
4	12350.0	309 days 20:49:00

```
# como queremos somente os dias...
rfm_p['Diff'] = rfm_p['Diff'].dt.days
rfm_p.head()
```

	CustomerID	Diff
0	12346.0	325
1	12347.0	1
2	12348.0	74
3	12349.0	18
4	12350.0	309

▼ Realizando a junção dos campos criados em uma nova base de dados

```
# utilizamos a função merge do pandas para realizar a mesclagem das bases através do atrib
dataset = pd.merge(rfm_m, rfm_f, on='CustomerID', how='inner')
dataset = pd.merge(dataset, rfm_p, on='CustomerID', how='inner')
dataset.columns = ['CustomerID', 'Amount', 'Frequency', 'Recency']
```

```
dataset.head()
```

	CustomerID	Amount	Frequency	Recency
0	12346.0	0.00	2	325
1	12347.0	4310.00	182	1
2	12348.0	1797.24	31	74
3	12349.0	1757.55	73	18
4	12350.0	334.40	17	309

▼ Identificando outliers

Para a realização do agrupamento é importante identificar se existem dados expurios que podem dificultar o processo de agrupamento

```
def print_outliers_distribution(data, attributes):  
    plt.rcParams['figure.figsize'] = [10,8]  
    sns.boxplot(data = dataset[attributes], orient="v", palette="Set2", whis=1.5, saturation=  
    plt.title("Dados expurios: Distribuição", fontsize = 14, fontweight = 'bold')  
    plt.ylabel("Amplitude", fontweight = 'bold')  
    plt.xlabel("Atributos", fontweight = 'bold')  
  
print_outliers_distribution(dataset, ['Amount', 'Frequency', 'Recency'])
```

► Removendo os dados expurios

Faremos a exclusão dos dados que podem atrapalhar a qualidade do agrupamento.

Hora da estatística!

Manteremos desta bases os dados que estão entre os quantis 5% e 95%.

```
[ ] ↳ 3 células ocultas
```

▼ Rescalando os Atributos

Para evitar que a amplitude dos atributos possa prejudicar o agrupamento, faremos a mudança da escala utilizando o StandardScaler

```
dados = dataset[['Amount', 'Frequency', 'Recency']]

from sklearn.preprocessing import StandardScaler

# Criamos o Scaler
scaler = StandardScaler()
# fit_transform
dados_rescalados = scaler.fit_transform(dados)

dataset_scaled = pd.DataFrame(dados_rescalados)
dataset_scaled.columns = ['Amount', 'Frequency', 'Recency']
dataset_scaled.head()
```

	Amount	Frequency	Recency
0	-0.723738	-0.752888	2.301611
1	1.731617	1.042467	-0.906466
2	0.300128	-0.463636	-0.183658
3	0.277517	-0.044720	-0.738141
4	-0.533235	-0.603275	2.143188

Etapa 4: Construindo o modelo

▼ K-means

O clustering K-means é um dos algoritmos de aprendizado de máquina não supervisionado mais simples e populares.

O algoritmo funciona da seguinte maneira:

1. Primeiro, inicializamos k pontos, chamados meios, aleatoriamente.
2. Categorizamos cada item para sua média mais próxima e atualizamos as coordenadas da média, que são as médias dos itens categorizados nessa média até agora.
3. Repetimos o processo para um determinado número de iterações e, ao final, temos nossos clusters.

Avaliaremos, também a qualidade dos clusters obtidos com o coeficiente de silhoueta

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

kmeans.labels_

array([1, 0, 2, ..., 1, 2, 2], dtype=int32)

ssd = []
sil = []
range_n_clusters = [2, 3, 4, 5, 6, 7, 8]
for num_clusters in range_n_clusters:
    kmeans = KMeans(n_clusters=num_clusters, max_iter=50)
    kmeans.fit(dataset_scaled)
    # silhouette score
    silhouette_avg = silhouette_score(dataset_scaled, kmeans.labels_)
    sil.append(silhouette_avg)
    ssd.append(kmeans.inertia_)

# plot the SSDs for each n_clusters
plt.plot(ssd)
ax = plt.gca()
ax.set_xticklabels(range_n_clusters)
```

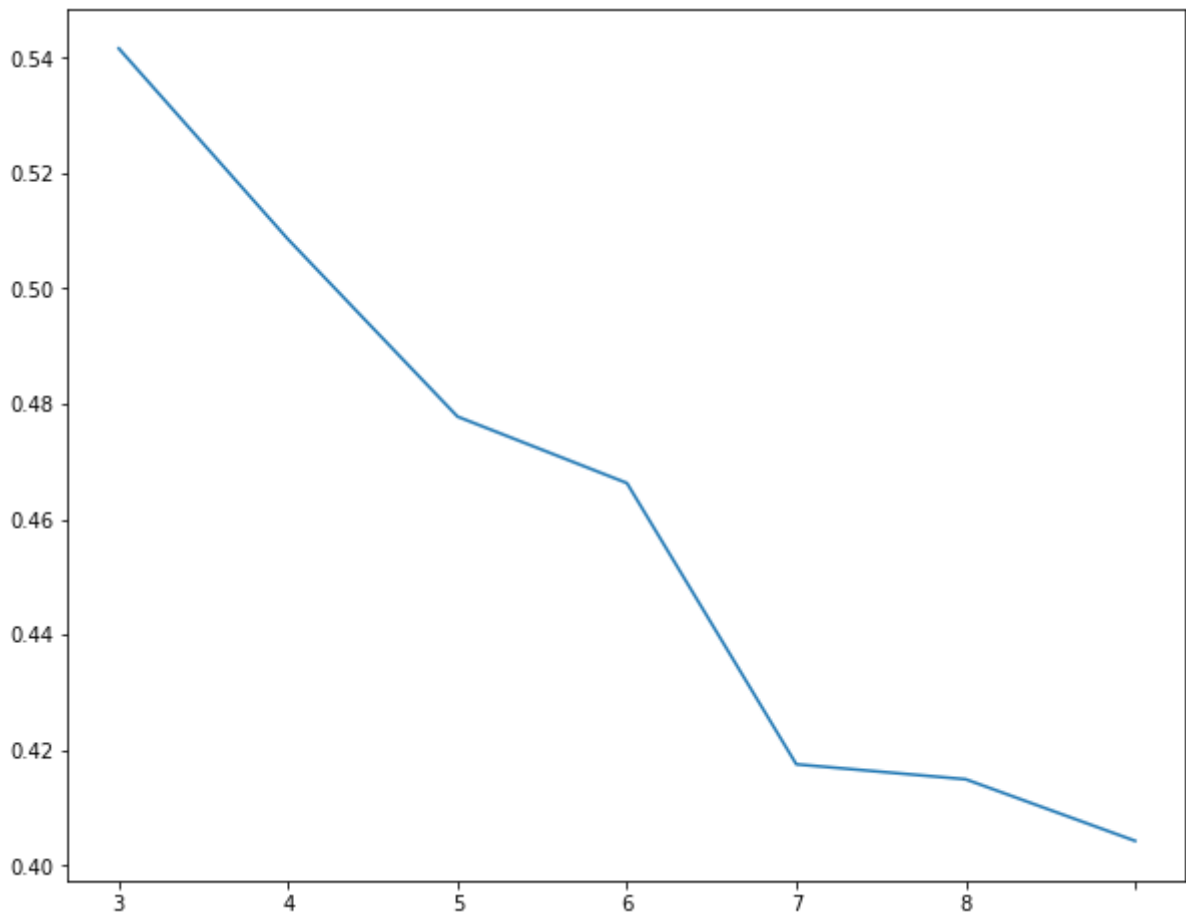


```
[Text(0, 0, '2'),  
Text(0, 0, '3'),  
Text(0, 0, '4'),  
Text(0, 0, '5'),  
Text(0, 0, '6'),  
Text(0, 0, '7'),  
Text(0, 0, '8')]
```



```
plt.plot(sil)  
ax = plt.gca()  
ax.set_xticklabels(range_n_clusters)
```

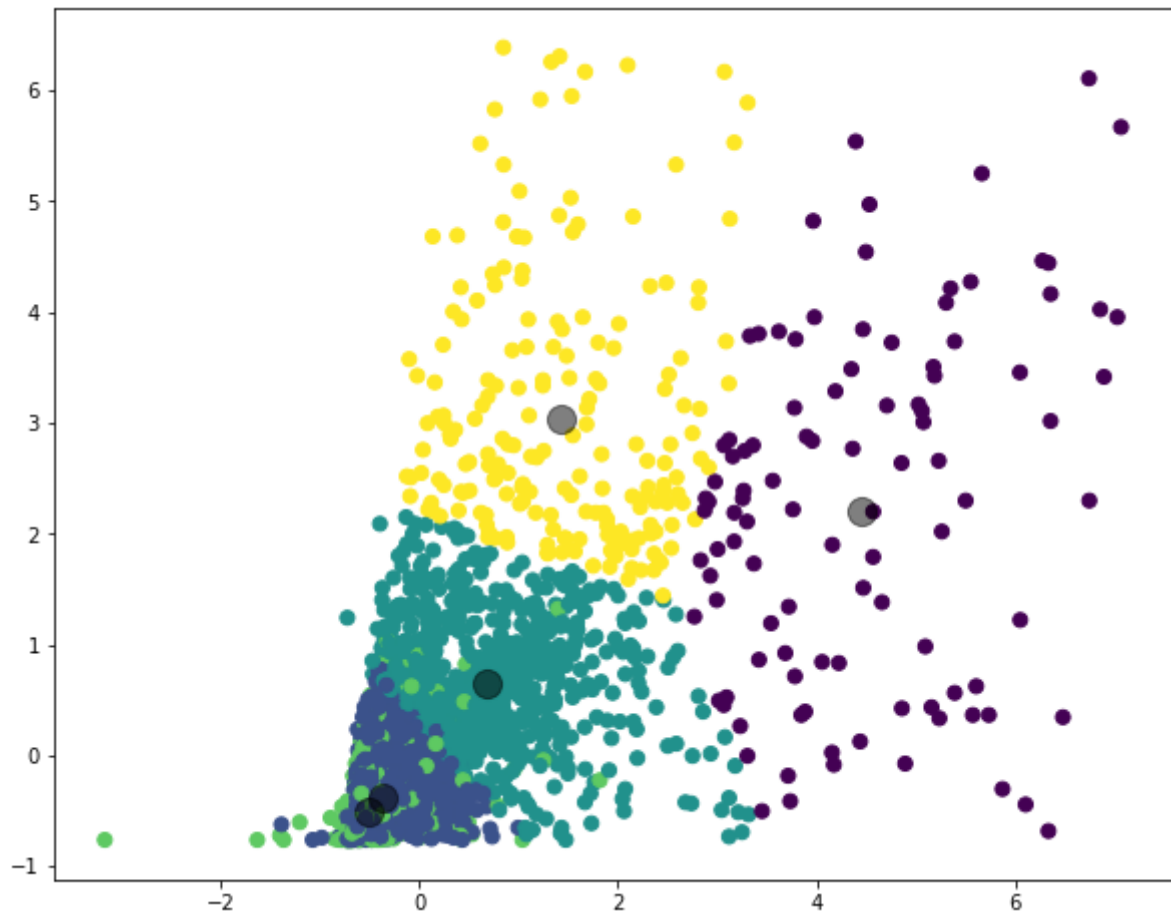
```
[Text(0, 0, '2'),  
Text(0, 0, '3'),  
Text(0, 0, '4'),  
Text(0, 0, '5'),  
Text(0, 0, '6'),  
Text(0, 0, '7'),  
Text(0, 0, '8')]
```



```
kmeans = KMeans(n_clusters=5)  
kmeans.fit(dataset_scaled)  
y_kmeans = kmeans.predict(dataset_scaled)
```

```
centers = kmeans.cluster_centers_
```

```
plt.scatter(dataset_scaled['Amount'], dataset_scaled['Frequency'], c=y_kmeans, s=50, cmap=
plt.scatter(centers[:, 0], centers[:, 1], c='black', s=200, alpha=0.5);
```

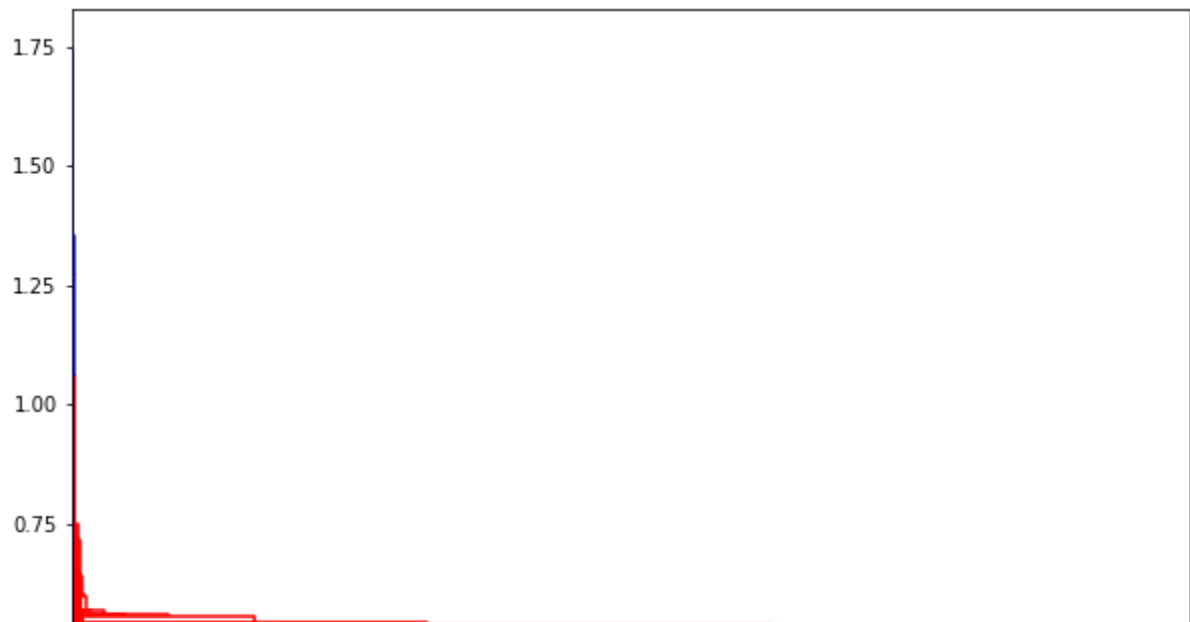


▼ CLustering Hierarquico

```
# Single linkage:
```

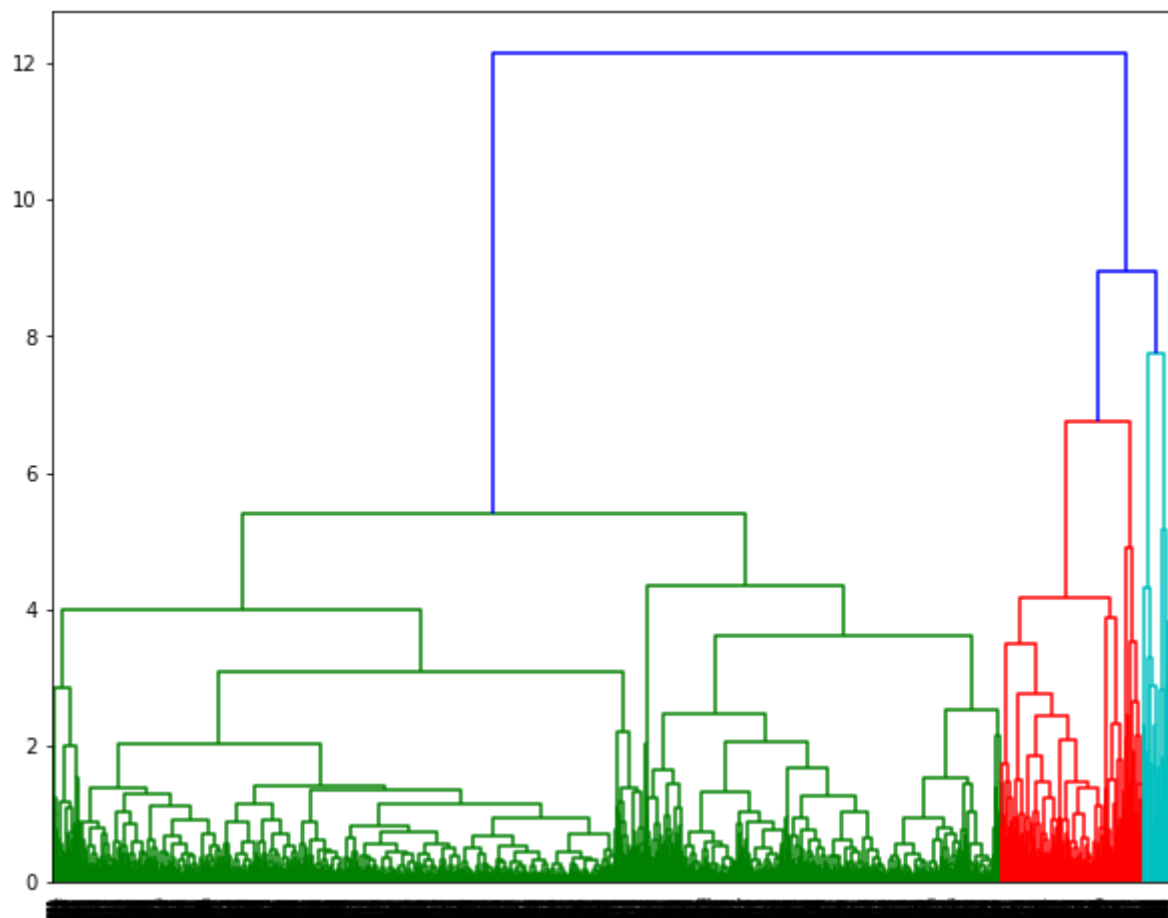
```
from scipy.cluster.hierarchy import linkage
from scipy.cluster.hierarchy import dendrogram
from scipy.cluster.hierarchy import cut_tree
```

```
mergings = linkage(dataset_scaled, method="single", metric='euclidean')
dendrogram(mergings)
plt.show()
```



Complete linkage

```
mergings = linkage(dataset_scaled, method="complete", metric='euclidean')
dendrogram(mergings)
plt.show()
```



Average linkage

```
mergings = linkage(dataset_scaled, method="average", metric='euclidean')
dendrogram(mergings)
plt.show()
```

```
# 3 clusters
cluster_labels = cut_tree(mergings, n_clusters=5).reshape(-1, )
cluster_labels

array([0, 1, 0, ..., 0, 0, 0])

dataset['Cluster_Labels'] = cluster_labels
dataset.head()
```

	CustomerID	Amount	Frequency	Recency	Cluster_Labels
0	12346.0	0.000000	2	325	0
1	12347.0	4310.000000	182	1	1
2	12348.0	1797.239990	31	74	0
3	12349.0	1757.550049	73	18	0
4	12350.0	334.399994	17	309	0

```
plt.scatter(dataset['Amount'], dataset['Frequency'], c=cluster_labels, s=50, cmap='viridis
```

